



Meu Primeiro App com o GitHub Copilot

Guia de prompts passo a passo — app de freelancers com avaliação 360°

Node (backend) + React (frontend) · CRUD, recrutamento, testes e documentação

Projeto Social Garapuvu 2026 · Instrutor: Douglas Adriano Queiroz

Este guia é executado ao vivo: você copia cada prompt no chat do GitHub Copilot (VS Code), na ordem apresentada, e mostra o sistema nascendo aos poucos. Ao final você terá uma plataforma de recrutamento de ponta a ponta — com CRUD completo pela interface, uma máquina de status do projeto, edição de perfil, testes automatizados em todos os níveis da pirâmide e testes end-to-end com dois usuários simultâneos.

✓ **Resultado esperado.** Sequência executada de ponta a ponta: o backend passa **26 testes** (unitários + integração); o frontend tem testes de componente e **2 testes E2E** (Playwright) — incluindo um smoke test do fluxo completo com dois atores (contratante + freelancer em janelas separadas). Cada tipo de teste pode ser rodado isoladamente por scripts npm dedicados.

1 Regras de ouro do Copilot

- **Dê contexto.** Deixe abertos os arquivos relevantes e use `@workspace` no Copilot Chat.
- **Peça em partes pequenas.** Um endpoint, um componente, um teste por vez.
- **Diga a stack e as regras.** "Node + Express", "status do projeto", "só o dono exclui".
- **Peça os testes junto.** Toda função/rota nova vem com teste.
- **Leia antes de aceitar.** Você assina o código. Não entendeu? `/explain`.
- **Teste e versione a cada passo.** Rode a suíte e faça `git commit` quando algo funcionar.
- **Seletores estáveis para E2E.** Ao pedir componentes, peça `data-testid` nos botões de ação (candidatar, selecionar, excluir, enviar-avaliação). É o que faz o robô do Playwright não quebrar quando o texto/visual muda — essencial para o smoke test de dois atores.

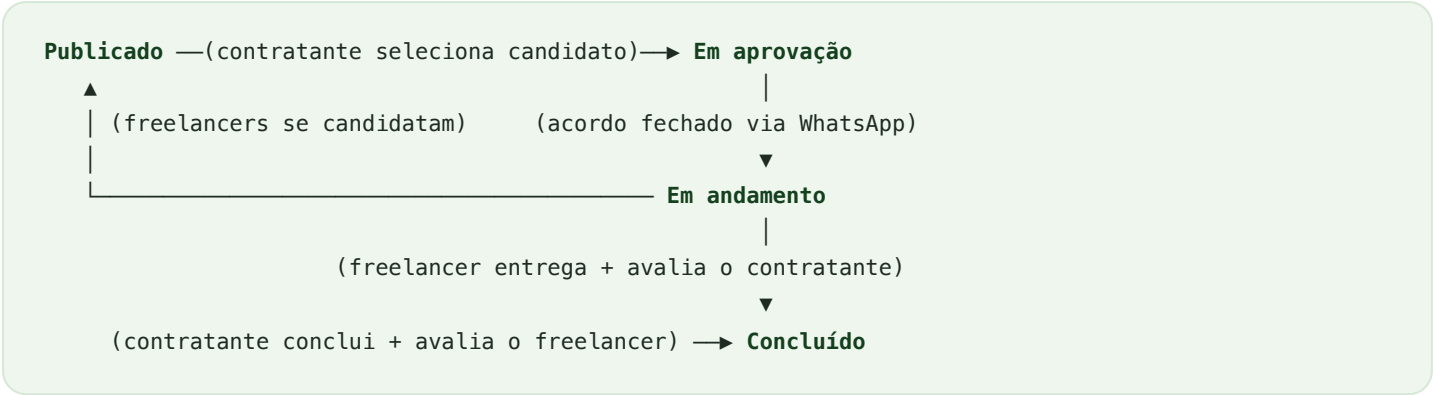
2 O app: FreelaAvalia 360 (recrutamento 360°)

Contratantes publicam projetos abertos; freelancers se candidatam; o contratante vê os candidatos com a reputação de cada um e seleciona; ao final, os dois se avaliam. A avaliação mútua (1 a 5 + comentário) é a **avaliação 360°**.

Entidade	O que guarda	Regras principais
Usuário	nome, e-mail, papel (freelancer/contratante), telefone e endereço (opcionais)	e-mail único; papel válido; senha ≥ 4 ; perfil editável

Entidade	O que guarda	Regras principais
Projeto	título, descrição, contratante, freelancer (opcional), contato, status	nasce aberto; percorre a máquina de status; só o dono edita/exclui
Candidatura	projeto + freelancer inscrito	1 por freelancer por projeto; só em projeto aberto
Avaliação	de quem, para quem, nota (1–5), comentário	durante o andamento ou após concluir; 1 por lado

A máquina de status do projeto



Regras do MVP: projeto em andamento não pode ser editado nem reaberto — para mudar, publique um novo. Editar/Excluir só aparecem enquanto o projeto está **Publicado**. A negociação de valores acontece no WhatsApp, fora do app; o contratante apenas registra no sistema quando fecha o acordo.

3 Bibliotecas e testes

Instale o Playwright dentro de `frontend/` :

```
# dentro de frontend/
npm i -D @playwright/test
npx playwright install chromium
```

A pirâmide de testes do projeto:

Nível	No FreelaAvalia 360	Arquivo
Unitário	validar nota, média, e-mail, <code>podeAvaliar</code>	<code>regras.test.js</code>
Componente	estrelas de nota	<code>Estrelas.test.jsx</code>
Integração	CRUD de projetos, fluxo de recrutamento, exclusão e avaliação 360	<code>app.test.js</code> (26 testes)
E2E	CRUD pela interface; fluxo completo com 2 atores	<code>projeto-crud.spec.js</code> , <code>fluxo-completo.spec.js</code>

4 As fases de base (0–6)

Antes das fases de recrutamento, o sistema é construído em seis fases:

- **Fase 0** — estrutura de pastas + README.
- **Fase 1** — regras puras (validação de nota, média, e-mail, `podeAvaliar`) + testes unitários.
- **Fase 2** — API Express (usuários, contratos, avaliações), CORS e login.
- **Fase 3** — frontend mobile-first (Landing, Auth, Estrelas, Avaliar, Perfil, Projetos em cards, modal "Novo projeto").
- **Fase 4** — testes de componente e E2E.
- **Fase 5** — cobertura + CI.
- **Fase 6** — documentação.

5 CRUD completo do projeto pela interface

PROMPT 5.1 — PROJETO ABERTO + CONTATO NO PERFIL

@workspace Ajuste o modelo de PROJETO para ser um anúncio ABERTO: o freelancer passa a ser opcional na criação (nasce `null = "Aguardando freelancer"`). Adicione ao usuário os campos OPCIONAIS endereço e telefone (no cadastro e no modelo). No POST /contratos, guarde descricao e o contato (email/endereco/telefone) do contratante – puxando do perfil quando não vierem no corpo. Atualize os testes de integração.

PROMPT 5.2 — EDITAR E EXCLUIR (PATCH E DELETE)

@workspace No backend, crie PATCH /contratos/:id (edita titulo/descricao/contato; título não pode ficar vazio) e DELETE /contratos/:id. No frontend, transforme o modal "Novo projeto" num formulário reutilizável (criar E editar): troque "ID do freelancer" por uma DESCRIÇÃO (textarea) e por telefone/endereço PRÉ-PREENCHIDOS do perfil (editáveis por projeto). Nos cards, mostre descrição e contato e adicione os botões Editar e Excluir (com confirmação) só para o dono.

PROMPT 5.3 — EDITAR O PRÓPRIO PERFIL

@workspace Crie PATCH /usuarios/:id (edita nome/telefone/endereco; nunca expõe a senha) e, na tela Perfil.jsx, além da reputação, adicione um formulário para editar nome, telefone e endereço. Ao salvar, atualize o usuário logado (e o localStorage) para que os próximos projetos já usem o contato novo. Associe cada label ao input com htmlFor/id (acessibilidade e testabilidade).

✓ **Verificar:** como contratante, crie um projeto, edite o título e exclua-o. Em "Meu perfil", altere o telefone e veja o novo valor sugerido ao criar o próximo projeto. **Boa prática:** a regra de quem-pode-o-quê também vale no backend — não basta esconder o botão.

6 Fluxo de recrutamento 360°

PROMPT 6.1 — CANDIDATURAS (FREELANCER SE INSCREVE)

@workspace Crie a entidade CANDIDATURA no repositório (projeto + freelancer) e os endpoints: POST /contratos/:id/candidaturas (só freelancer, projeto aberto, sem duplicar) e GET /contratos/:id/candidaturas (lista os candidatos com nome e a MÉDIA das avaliações 360). Inclua os ids dos candidatos no GET /contratos para o front montar os botões.

PROMPT 6.2 — MÁQUINA DE STATUS (SELEÇÃO → ANDAMENTO → CONCLUSÃO)

@workspace Implemente as transições, validando a ordem:

- PATCH /contratos/:id/selecionar {freelancerId}: só entre os candidatos → status "em_aprovacao";
- PATCH /contratos/:id/andamento: confirma o acordo (fechado no WhatsApp) → "em_andamento";
- PATCH /contratos/:id/concluir: só a partir de "em_andamento" → "concluido".

Ajuste podeAvaliar para liberar a avaliação em "em_andamento" e "concluido". Cubra tudo com testes de integração (inclusive as ordens inválidas devolvendo 400).

PROMPT 6.3 — TELAS POR PAPEL

@workspace Em Projetos.jsx, mostre ações conforme o papel e o status:

- Freelancer: vê os projetos abertos e o botão "Candidatar-se"; quando selecionado e o projeto em andamento, "Finalizar trabalho e enviar feedback" (avaliação 360 para o contratante).
- Contratante: "Ver candidatos (N)" abre um modal com a reputação (estrelas) e "Selecionar"; depois "Fechei o acordo → iniciar" e, quando o freelancer entregar, "Concluir e avaliar freelancer".

Use data-testid nos botões de ação.

PROMPT 6.4 — REGRAS DE EXCLUSÃO DE PROJETO

@workspace Refine o DELETE /contratos/:id: projeto CONCLUÍDO nunca pode ser excluído; só projetos PUBLICADOS (aberto) podem ser excluídos; se houver candidatos inscritos, é preciso removê-los antes. Crie DELETE /contratos/:id/candidaturas/:freelancerId para retirar a candidatura (pelo próprio freelancer OU pelo contratante). No front, adicione "Retirar candidatura" para o freelancer e "Remover" para cada candidato no modal. Teste as três regras.

✓ **Verificar:** abra duas janelas (uma logada como contratante, outra como freelancer). Publique → candidate-se → selecione → inicie → freelancer avalia → contratante conclui e avalia. Tente excluir um projeto concluído: deve ser bloqueado com mensagem clara.

7 Smoke test E2E com dois atores

O smoke test percorre o caminho crítico do sistema de ponta a ponta. Ele usa dois contextos de navegador (contratante e freelancer logados ao mesmo tempo) e pode rodar em modo **headed** com **slowMo** alto, para dar pra acompanhar cada interação na tela durante a aula.

PROMPT 7.1 — CONFIG COM SLOWMO POR VARIÁVEL DE AMBIENTE

@workspace Em frontend/playwright.config.js, aponte baseUrl para http://localhost:5173 e configure use.launchOptions.slowMo lendo process.env.SLOWMO (0 por padrão). Assim o mesmo teste roda rápido no CI (headless) e devagar na aula (headed).

PROMPT 7.2 — SMOKE TEST DO FLUXO COMPLETO (2 ATORES)

@workspace Crie frontend/e2e/fluxo-completo.spec.js. Use { browser } e crie DOIS contextos (contratante e freelancer). Passos: contratante cadastra e publica a vaga → freelancer cadastra, vê a vaga e se candidata → contratante recarrega, abre "Ver candidatos", seleciona → "Fechei o acordo → iniciar" → freelancer recarrega, "Finalizar trabalho", dá 5 estrelas e envia → contratante recarrega, "Concluir e avaliar", 5 estrelas e envia → verifica o status "Concluído". Use getByTestId nos botões e recarregue a página ao trocar de ator (dados em memória).

PROMPT 7.3 — SCRIPT NPM DEDICADO

@workspace No package.json do frontend, adicione:
"e2e:smoke": "SLOWMO=900 playwright test fluxo-completo --headed --workers=1".
Assim eu rodo SÓ o fluxo completo, devagar e com o navegador visível.

PROMPT 7.4 — WEBSERVER AUTOMÁTICO (SOBE BACKEND E FRONTEND SOZINHO)

@workspace Em frontend/playwright.config.js, adicione a chave webServer com DOIS serviços: backend (command "npm start", cwd "../backend", url http://localhost:3001/contratos) e frontend (command "npm run dev", url http://localhost:5173), ambos com reuseExistingServer: !process.env.CI e timeout 30000. Assim os testes E2E sobem e aguardam os serviços sozinhos, sem depender de subir tudo à mão (evita o timeout de 30s quando um serviço não está no ar).

```
# com o webServer configurado, basta:  
cd frontend  
npm run e2e          # todos os E2E (headless) – sobe os serviços sozinho  
npm run e2e:smoke    # só o fluxo completo, headed + slowMo
```

Resultado: backend 26 testes, frontend 3 de componente e 2 E2E passando. O smoke test abre duas janelas e mostra o projeto indo de Publicado a Concluído com as duas avaliações.

8 Tags e scripts por tipo de teste

Para apresentar a pirâmide na aula — e para rodar cada nível isoladamente —, cada teste recebe uma **tag** no título. A tag serve tanto para `grep` nos arquivos quanto para filtrar a execução (`vitest -t` / `playwright --grep`).

PROMPT 8.1 — MARCAR TESTES POR TIPO E CRIAR OS SCRIPTS

@workspace Marque os testes por tipo para permitir filtrar por grep e por execução:
– prefixe os describe com "@unitario" (regras.test.js) e "@integracao" (app.test.js);
– use tag ["@interface", "@e2e"] nos testes do Playwright e "@interface" no describe de Estrelas.test.jsx.
No package.json do backend, adicione "test:unit": "vitest run -t @unitario" e "test:integracao": "vitest run -t @integracao". No package.json do frontend, adicione "test:interface": "vitest run -t @interface", "e2e:interface": "playwright test --grep @interface" e "e2e:e2e": "playwright test --grep @e2e".

Tag	Tipo de teste	Onde vive
@unitario	regras de negócio	backend/src/regras.test.js
@integracao	API (Supertest)	backend/src/app.test.js
@interface	componente (RTL) + E2E	Estrelas.test.jsx + specs E2E
@e2e	só os E2E de navegador	frontend/e2e/*.spec.js

Rodar por tipo

```
# backend
npm run test:unit      # só regras de negócio
npm run test:integracao # só a API

# frontend
npm run test:interface # só componente
npm run e2e:e2e        # só E2E de navegador
```

Identificar por grep

```
grep -rl "@unitario" backend/ # arquivos de teste unitário
grep -rl "@integracao" backend/ # integração
grep -rln "@e2e" frontend/ # E2E de navegador
```

9 Endpoints da API

Método	Rota	Descrição
POST	/usuarios	Cadastro (endereço/telefone opcionais)
POST	/login	Autentica (retorna o usuário sem a senha)
GET	/usuarios/:id	Usuário + média das avaliações
PATCH	/usuarios/:id	Edita o próprio perfil
POST	/contratos	Publica um projeto aberto
GET	/contratos	Lista projetos (com candidatos e avaliadores)
PATCH	/contratos/:id	Edita título/descrição/contato
DELETE	/contratos/:id	Exclui (só publicado e sem candidatos)
POST	/contratos/:id/candidaturas	Freelancer se candidata
GET	/contratos/:id/candidaturas	Candidatos com reputação
DELETE	/contratos/:id/candidaturas/:fid	Retira candidatura
PATCH	/contratos/:id/selecionar	Seleciona candidato → Em aprovação
PATCH	/contratos/:id/andamento	Confirma acordo → Em andamento
PATCH	/contratos/:id/concluir	Encerra → Concluído
POST	/avaliacoes	Avaliação 360 (em andamento/concluído)

10

Conceitos-chave do projeto

Conceito	O que é	Onde aparece
Máquina de estados	Status que só avançam numa ordem válida	<code>app.js</code> (selecionar/andamento/concluir)
Regras de autorização	Quem pode excluir/editar e quando	DELETE guard + botões por papel
Seletores estáveis	<code>data-testid</code> resistente a mudança de visual	botões de ação
CRUD via interface	Criar, ler, editar e excluir sem tocar no banco	<code>Projetos.jsx</code> + PATCH/DELETE
Multi-ator no E2E	Dois contextos de navegador no mesmo teste	<code>fluxo-completo.spec.js</code>
WebServer no Playwright	Sobe e aguarda os serviços automaticamente	<code>playwright.config.js</code> (webServer)
Tags de teste / greps	Rodar e identificar cada nível da pirâmide	títulos dos testes + scripts npm

Para apresentar aos alunos

Rode a **pirâmide viva** — unitários (instantâneos), integração (26 testes) e, por fim, `npm run e2e:smoke` : o navegador abre e o projeto percorre todo o ciclo em duas janelas. É o melhor "final de aula": o sistema inteiro funcionando diante deles.